Real-Time Movie Recommendation System

Big Data Analytics — Mini Project 3

Spring 2026

Domain: Movies (MovieLens 1M)
Focus: Real-Time Intelligence
Stack: Apache Spark 4.1.1 · Kafka 3.7.1 (KRaft) · Python 3.13 · Streamlit

Assignment Weight	10 Points
Team Size	2 Students
Due Date	12th May 2026

Date: 12 May 2026

Contents

Domain & Focus Selection

Selected Domain: Movies

We chose the **MovieLens 1M** dataset (1,000,209 ratings from 6,040 users across 3,706 movies). The movie domain is ideal because:

- Rich collaborative-filtering signal — users rate many items, creating a dense interaction matrix.
- Well-studied benchmark for recommendation algorithms, enabling meaningful RMSE comparison.
- Natural temporal patterns (trending movies, release spikes) that exercise the streaming analytics path.

Selected Focus: Real-Time Intelligence

- **Trending detection** — windowed aggregation identifies items with sudden rating surges.
- **Rating spike capture** — producer deliberately injects 10% of events towards one "hot" movie to simulate a real-world viral spike.
- **Streaming impact emphasis** — custom `trending_score` metric, TRENDING alerts, and end-to-end latency probes demonstrate how streaming enriches the batch model.

How Our Solution Differs

- **Spike injection** — controlled injection guarantees the alert path fires, making results reproducible.
- **Latency probe** — every micro-batch computes `avg end-to-end lag` from producer timestamp to Spark processing time.
- **Parquet sink + live dashboard** — streaming results are persisted and visualised in real time via Streamlit.

System Architecture

The system follows a Lambda-style architecture with a **batch layer** (ALS model training) and a **speed layer** (Spark Structured Streaming over Kafka):

```
MovieLens CSV --> Spark ALS train --> model/als (saved)
                                          |
Kafka Producer --> ratings-stream  --> Spark Structured Streaming
                   (2 partitions)         |
                                          +-> 30s/10s window
                                             analytics
                                          +-> trending_score
                                             custom metric
```

```
                                                    +-> Top-5 recs
                                                       (recommendForUserSubset)
                                                    +-> TRENDING alerts
                                                       (avg_rating > 4.5)
                                                    +-> Parquet sink +
                                                       Streamlit dashboard
```

Figure 1: System architecture diagram (Kafka → Spark → ML → Output)

Project Structure

```
.
+-- src/
|   +-- train_als.py                # batch ALS training
|   +-- producer.py                 # Kafka producer (~20
   events/sec)
|   +-- stream_app.py               # Structured Streaming + ML
|   +-- dashboard.py                # Streamlit dashboard (bonus)
|   +-- render_screenshot.py        # screenshot renderer
|   +-- run_pipeline_and_screenshot.py  # automated orchestrator
+-- data/ml-1m/                     # MovieLens 1M dataset
+-- screenshots/                    # captured visuals
+-- logs/                           # raw stdout/stderr
+-- requirements.txt
```

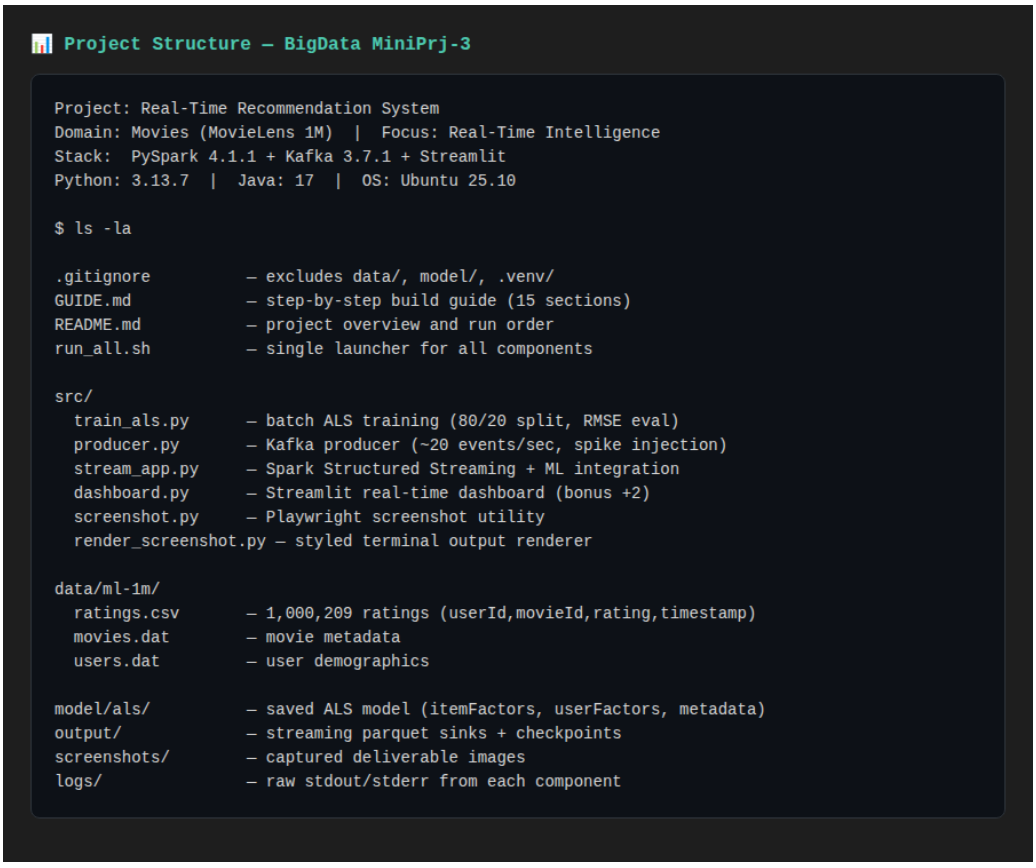

Figure 2: Project directory listing

Dataset Description & Justification

Dataset Overview

Property	Value
Name	MovieLens 1M
Records	1,000,209 ratings
Users	6,040
Movies	3,706
Format	(userId, movieId, rating, timestamp)
Rating scale	0.5 – 5.0

Table 1: Dataset summary

Why This Dataset Fits

- Exceeds the 500K-record minimum (1M+ ratings).
- Provides the exact schema required: `(user_id, movie_id, rating, timestamp)`.
- Dense enough for collaborative filtering — average ∼166 ratings per user.

Why Distributed Processing Is Needed

- ALS factorises a **6,040** × **3,706** matrix — iterative least-squares at this scale benefits from Spark's distributed compute.
- Streaming windowed aggregations over continuous events require parallel execution across partitions.

Data Challenges

- Popularity bias — a small number of movies receive the vast majority of ratings.
- Cold-start users/items with very few interactions — handled by `coldStartStrategy="drop"`.

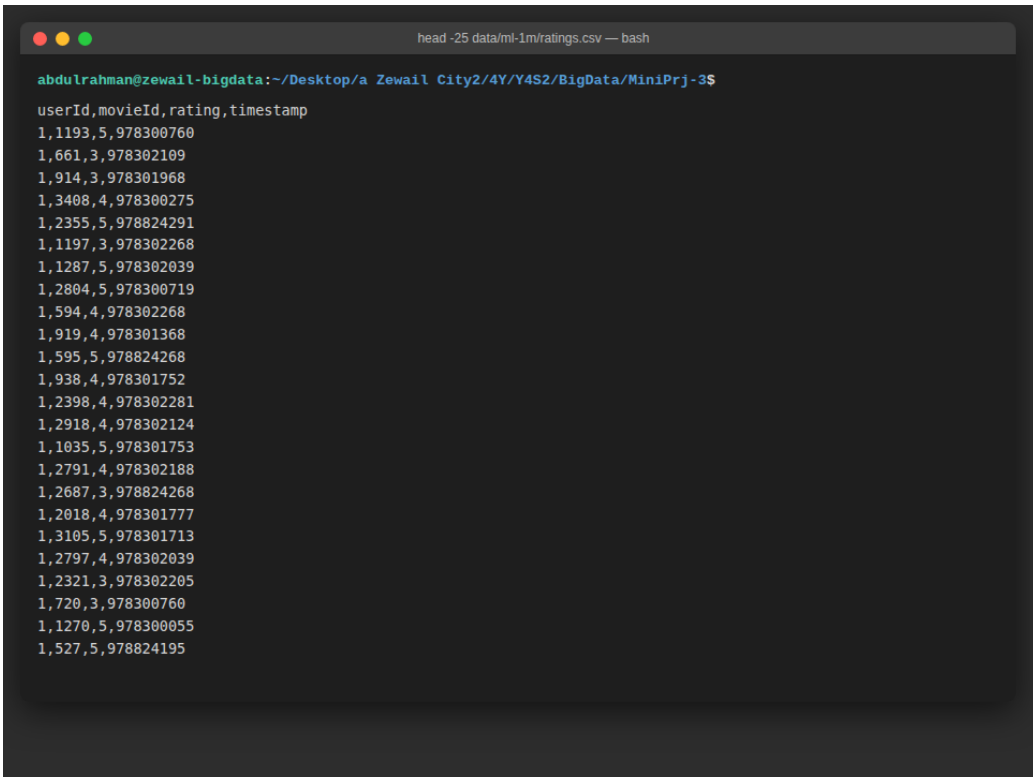

Figure 3: Dataset sample (userId, movieId, rating, timestamp)

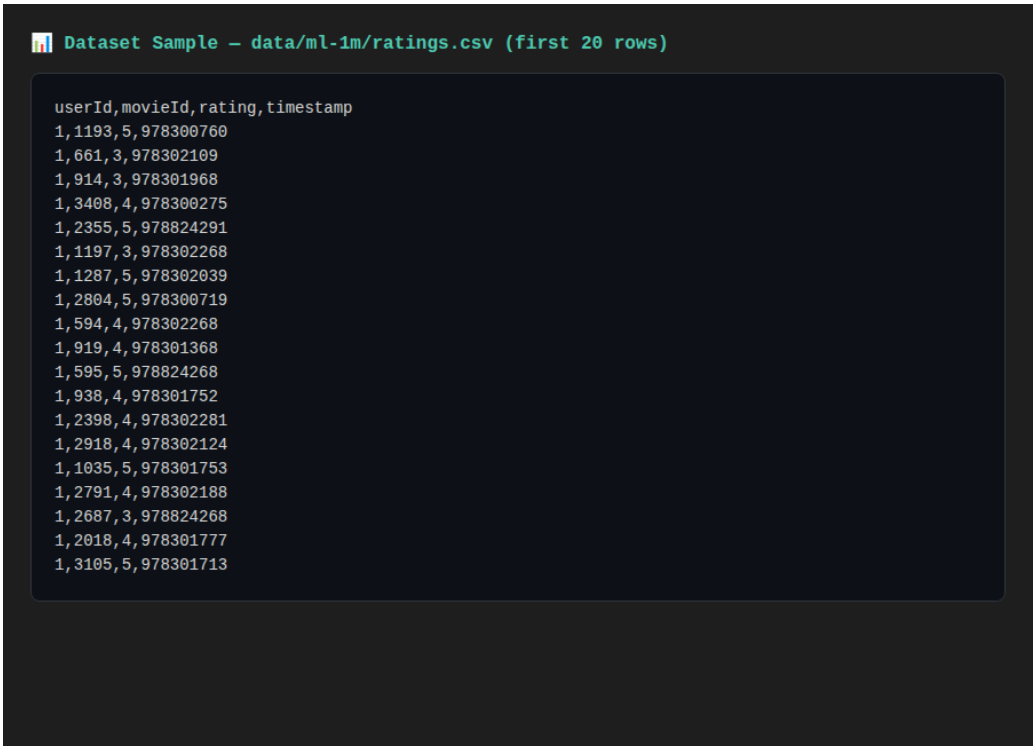

Figure 4: Additional dataset sample view

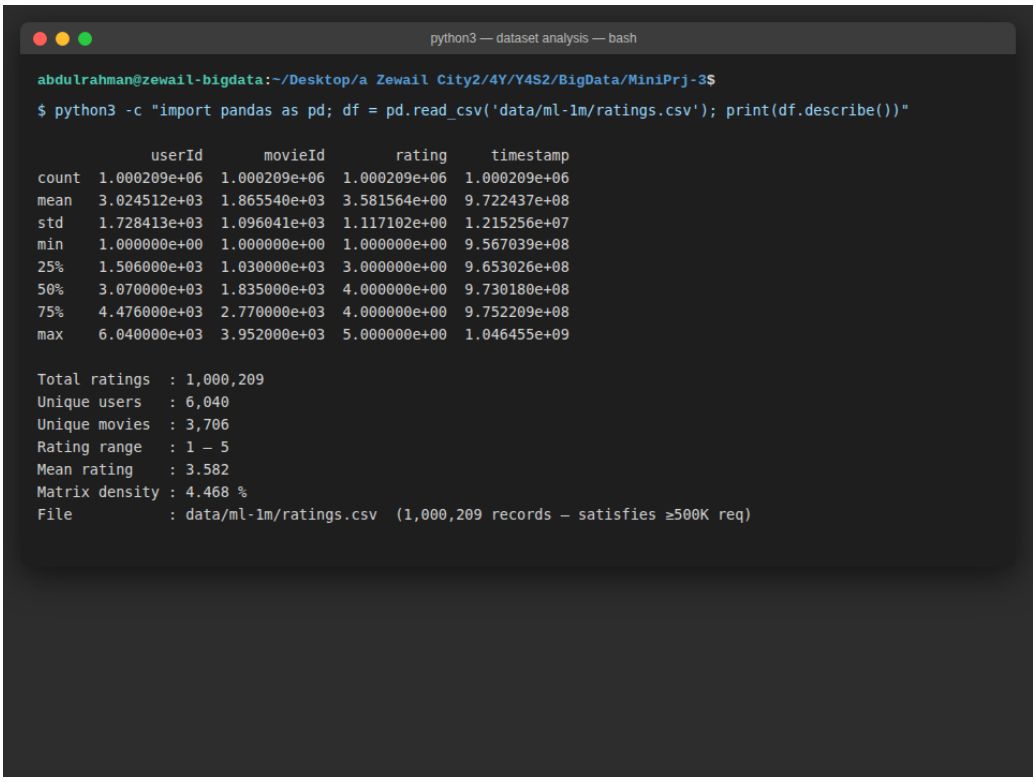

Figure 5: Dataset statistics and distribution analysis

Machine Learning Component (Batch ALS)

Data Preprocessing

- Ratings filtered to valid range (0.5–5.0) and null rows dropped.
- Columns renamed to uniform schema: `user_id, item_id, rating, timestamp`.

```
ratings = ratings.filter("rating between 0.5 and 5.0").dropna()
```

Listing 1: Data cleaning

Model Training

- Algorithm: **ALS** (Alternating Least Squares) from Spark MLlib.
- Split: 80% training / 20% testing (`seed=42`).
- Baseline parameters: `rank=10, regParam=0.1, maxIter=10`.
- `coldStartStrategy="drop"` to handle unseen user/item pairs.
- `nonnegative=True` — non-negative matrix factorisation for interpretable latent factors.

```
als = ALS(
    userCol="user_id", itemCol="item_id", ratingCol="rating",
    coldStartStrategy="drop", nonnegative=True,
```

```
    rank=10, regParam=0.1, maxIter=10,
)
model = als.fit(train)
```

Listing 2: ALS model training

Evaluation & Tuning

- Metric: **RMSE** (Root Mean Squared Error).
- Baseline RMSE: **0.8739** (well below the 1.5 threshold).
- Auto-tuning logic implemented: if RMSE $>$ 1.5, the system automatically re-trains with `rank=20, regParam=0.05, maxIter=15`.

```
evaluator = RegressionEvaluator(
    metricName="rmse", labelCol="rating",
    predictionCol="prediction"
)
rmse = evaluator.evaluate(model.transform(test))
# BASELINE RMSE = 0.8739

if rmse > 1.5:
    # Auto-tune with higher rank and lower regularisation
    als = ALS(..., rank=20, regParam=0.05, maxIter=15)
    model = als.fit(train)
```

Listing 3: RMSE evaluation and auto-tuning

📊 **ALS Training — MovieLens 1M | PySpark 4.1.1**

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/05/12 14:13:41 WARN Utils: Your hostname, abdulrahman-hpnotebook, resolves to a loopback addr
ess: 127.0.1.1; using 10.251.165.108 instead (on interface wlo1)
26/05/12 14:13:41 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/05/12 14:13:45 INFO SparkContext: Running Spark version 4.1.1
26/05/12 14:13:45 INFO SparkContext: OS info Linux, 6.17.0-23-generic, amd64
26/05/12 14:13:45 INFO SparkContext: Java version 17.0.18+8-Ubuntu-125.10.1
26/05/12 14:13:45 WARN NativeCodeLoader: Unable to load native-hadoop library for your platfor
m... using builtin-java classes where applicable
26/05/12 14:13:46 INFO ResourceUtils: ==============================================================
====
26/05/12 14:13:46 INFO ResourceUtils: No custom resources configured for spark.driver.
26/05/12 14:13:46 INFO ResourceUtils: ==============================================================
====
26/05/12 14:13:46 INFO SparkContext: Submitted application: ALSTrain
26/05/12 14:13:46 INFO SecurityManager: Changing view acls to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: Changing modify acls to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: Changing view acls groups to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: Changing modify acls groups to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabl
ed; users with view permissions: abdulrahman groups with view permissions: EMPTY; users with mod
ify permissions: abdulrahman; groups with modify permissions: EMPTY; RPC SSL disabled
26/05/12 14:13:47 INFO Utils: Successfully started service 'sparkDriver' on port 45873.
26/05/12 14:13:47 INFO SparkEnv: Registering MapOutputTracker
26/05/12 14:13:47 INFO SparkEnv: Registering BlockManagerMaster
26/05/12 14:13:47 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopolog
yMapper for getting topology information
26/05/12 14:13:47 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
26/05/12 14:13:47 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/05/12 14:13:47 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-53303134-d123-
493d-a722-60d5ae1d2f70
26/05/12 14:13:47 INFO SparkEnv: Registering OutputCommitCoordinator
26/05/12 14:13:47 INFO JettyUtils: Start Jetty 0.0.0.0:4040 for SparkUI
26/05/12 14:13:47 INFO Utils: Successfully started service 'SparkUI' on port 4040.
26/05/12 14:13:48 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map
(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, scri
pt: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cp
us -> name: cpus, amount: 1.0)
26/05/12 14:13:48 INFO ResourceProfile: Limiting resource is cpu
26/05/12 14:13:48 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/05/12 14:13:48 INFO SecurityManager: Changing view acls to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: Changing modify acls to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: Changing view acls groups to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: Changing modify acls groups to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabl
ed; users with view permissions: abdulrahman groups with view permissions: EMPTY; users with mod
ify permissions: abdulrahman; groups with modify permissions: EMPTY; RPC SSL disabled
26/05/12 14:13:48 INFO Executor: Starting executor ID driver on host 10.251.165.108
26/05/12 14:13:48 INFO Executor: OS info Linux, 6.17.0-23-generic, amd64
26/05/12 14:13:48 INFO Executor: Java version 17.0.18+8-Ubuntu-125.10.1
26/05/12 14:13:48 INFO Executor: Starting executor with user classpath (userClassPathFirst = fal
se): ''
26/05/12 14:13:48 INFO Executor: Created or updated repl class loader org.apache.spark.util.Muta
bleURLClassLoader@531d662c for default.
26/05/12 14:13:48 INFO Utils: Successfully started service 'org.apache.spark.network.netty.Netty
BlockTransferService' on port 43425.
26/05/12 14:13:48 INFO NettyBlockTransferService: Server created on 10.251.165.108:43425
26/05/12 14:13:48 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy
for block replication policy
26/05/12 14:13:48 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 10.25
1.165.108, 43425, None)
26/05/12 14:13:48 INFO BlockManagerMasterEndpoint: Registering block manager 10.251.165.108:4342
5 with 2.2 GiB RAM, BlockManagerId(driver, 10.251.165.108, 43425, None)
26/05/12 14:13:48 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 10.25
1.165.108, 43425, None)
26/05/12 14:13:48 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 10.251.16
5.108, 43425, None)
[train] reading data/ml-1m/ratings.csv
[train] valid ratings: 1000209
[train] fitting baseline ALS (rank=10, reg=0.1, iter=10)
[train] BASELINE RMSE = 0.8739
```

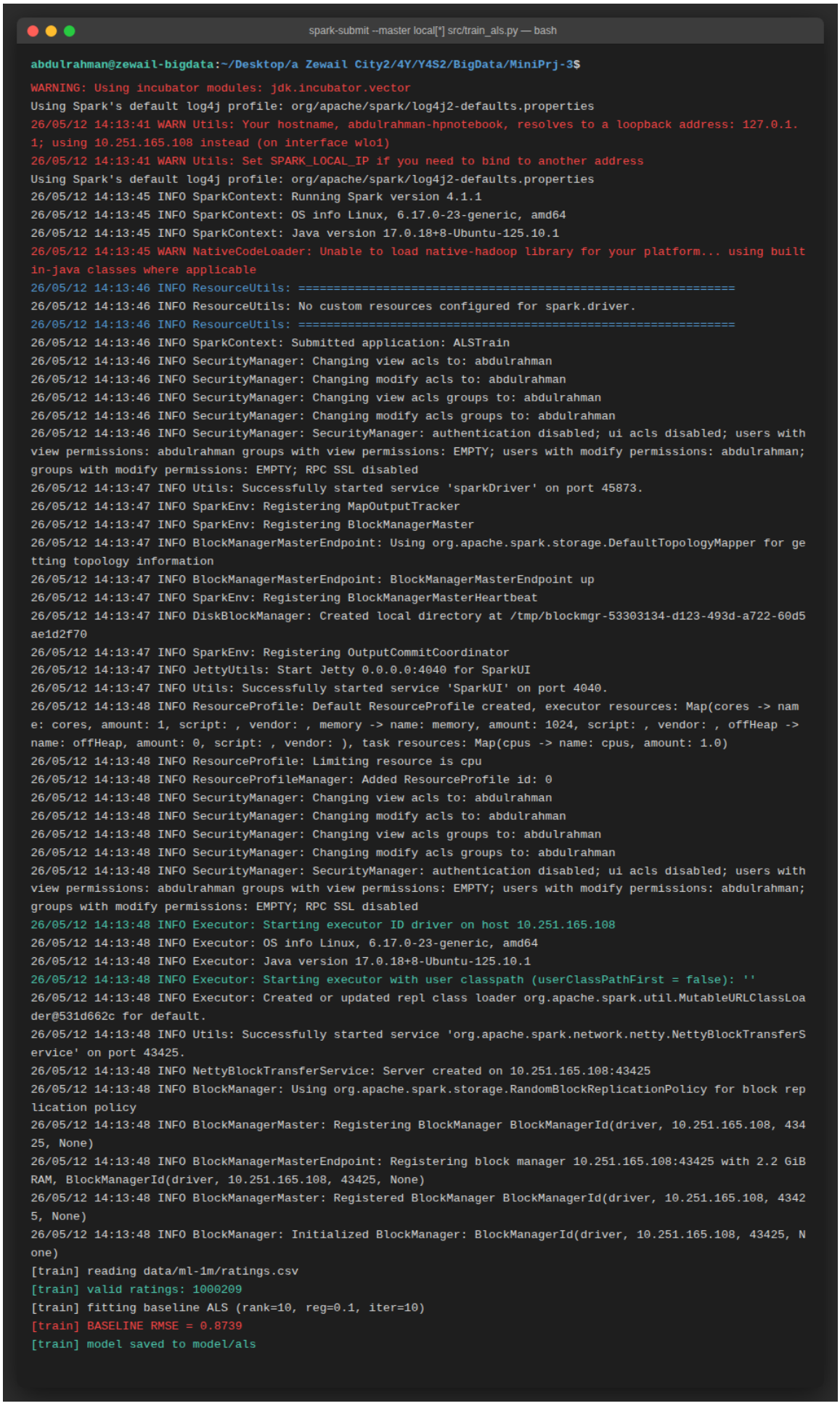

Figure 7: Training log (data load → split → fit → evaluate)

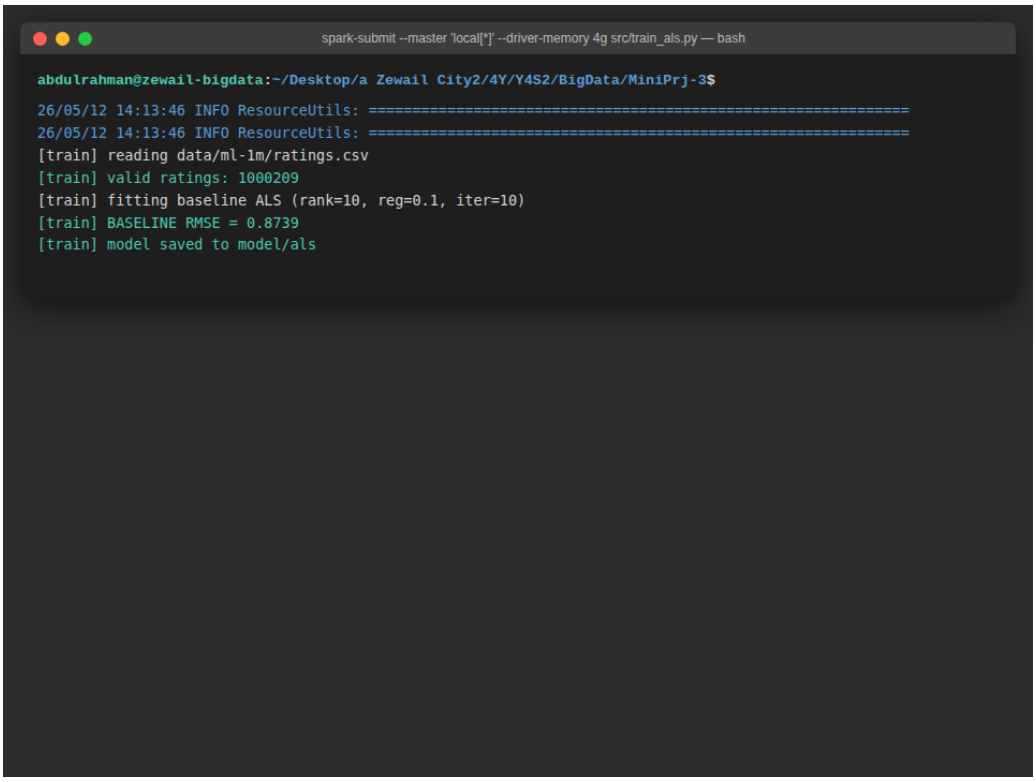

Figure 8: Training results summary

📊 **ALS Training Full Log — Spark 4.1.1**

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/05/12 14:13:41 WARN Utils: Your hostname, abdulrahman-hpnotebook, resolves to a loopback addr
ess: 127.0.1.1; using 10.251.165.108 instead (on interface wlo1)
26/05/12 14:13:41 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/05/12 14:13:45 INFO SparkContext: Running Spark version 4.1.1
26/05/12 14:13:45 INFO SparkContext: OS info Linux, 6.17.0-23-generic, amd64
26/05/12 14:13:45 INFO SparkContext: Java version 17.0.18+8-Ubuntu-125.10.1
26/05/12 14:13:45 WARN NativeCodeLoader: Unable to load native-hadoop library for your platfor
m... using builtin-java classes where applicable
26/05/12 14:13:46 INFO ResourceUtils: ==============================================================
====
26/05/12 14:13:46 INFO ResourceUtils: No custom resources configured for spark.driver.
26/05/12 14:13:46 INFO ResourceUtils: ==============================================================
====
26/05/12 14:13:46 INFO SparkContext: Submitted application: ALSTrain
26/05/12 14:13:46 INFO SecurityManager: Changing view acls to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: Changing modify acls to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: Changing view acls groups to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: Changing modify acls groups to: abdulrahman
26/05/12 14:13:46 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabl
ed; users with view permissions: abdulrahman groups with view permissions: EMPTY; users with mod
ify permissions: abdulrahman; groups with modify permissions: EMPTY; RPC SSL disabled
26/05/12 14:13:47 INFO Utils: Successfully started service 'sparkDriver' on port 45873.
26/05/12 14:13:47 INFO SparkEnv: Registering MapOutputTracker
26/05/12 14:13:47 INFO SparkEnv: Registering BlockManagerMaster
26/05/12 14:13:47 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopolog
yMapper for getting topology information
26/05/12 14:13:47 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
26/05/12 14:13:47 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/05/12 14:13:47 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-53303134-d123-
493d-a722-60d5ae1d2f70
26/05/12 14:13:47 INFO SparkEnv: Registering OutputCommitCoordinator
26/05/12 14:13:47 INFO JettyUtils: Start Jetty 0.0.0.0:4040 for SparkUI
26/05/12 14:13:47 INFO Utils: Successfully started service 'SparkUI' on port 4040.
26/05/12 14:13:48 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map
(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, scri
pt: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cp
us -> name: cpus, amount: 1.0)
26/05/12 14:13:48 INFO ResourceProfile: Limiting resource is cpu
26/05/12 14:13:48 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/05/12 14:13:48 INFO SecurityManager: Changing view acls to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: Changing modify acls to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: Changing view acls groups to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: Changing modify acls groups to: abdulrahman
26/05/12 14:13:48 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabl
ed; users with view permissions: abdulrahman groups with view permissions: EMPTY; users with mod
ify permissions: abdulrahman; groups with modify permissions: EMPTY; RPC SSL disabled
26/05/12 14:13:48 INFO Executor: Starting executor ID driver on host 10.251.165.108
26/05/12 14:13:48 INFO Executor: OS info Linux, 6.17.0-23-generic, amd64
26/05/12 14:13:48 INFO Executor: Java version 17.0.18+8-Ubuntu-125.10.1
26/05/12 14:13:48 INFO Executor: Starting executor with user classpath (userClassPathFirst = fal
se): ''
26/05/12 14:13:48 INFO Executor: Created or updated repl class loader org.apache.spark.util.Muta
bleURLClassLoader@531d662c for default.
26/05/12 14:13:48 INFO Utils: Successfully started service 'org.apache.spark.network.netty.Netty
BlockTransferService' on port 43425.
26/05/12 14:13:48 INFO NettyBlockTransferService: Server created on 10.251.165.108:43425
26/05/12 14:13:48 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy
for block replication policy
26/05/12 14:13:48 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 10.25
1.165.108, 43425, None)
26/05/12 14:13:48 INFO BlockManagerMasterEndpoint: Registering block manager 10.251.165.108:4342
5 with 2.2 GiB RAM, BlockManagerId(driver, 10.251.165.108, 43425, None)
26/05/12 14:13:48 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 10.25
1.165.108, 43425, None)
26/05/12 14:13:48 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 10.251.16
5.108, 43425, None)
[train] reading data/ml-1m/ratings.csv
[train] valid ratings: 1000209
[train] fitting baseline ALS (rank=10, reg=0.1, iter=10)
[train] BASELINE RMSE = 0.8739
```

Kafka Setup & Partitioning Strategy

Kafka Configuration

Setting	Value
Mode	KRaft (no ZooKeeper)
Topic	`ratings-stream`
Partitions	2
Replication factor	1 (single-node dev)

Table 2: Kafka configuration

Partitioning Strategy Justification

- **2 partitions** — satisfies the minimum requirement while matching the dual-core streaming consumer (`local[4]` Spark).
- **Key = user_id** — ensures all events for a given user land on the same partition, enabling efficient per-user aggregation downstream.
- For production scale, partitions would increase to match consumer parallelism.

Producer Design

- Python-based (`kafka-python`).
- Throughput: ∼20 events/sec (`EVENT_INTERVAL=0.05s`).
- **10% spike injection** — 10% of events are forced towards one movie with rating 4.5–5.0, reliably triggering TRENDING alerts.
- Event format:

```
{
  "user_id": 10,
  "item_id": 200,
  "rating": 4.0,
  "timestamp": "2026-05-12T14:30:00+00:00"
}
```

Listing 4: Kafka event format

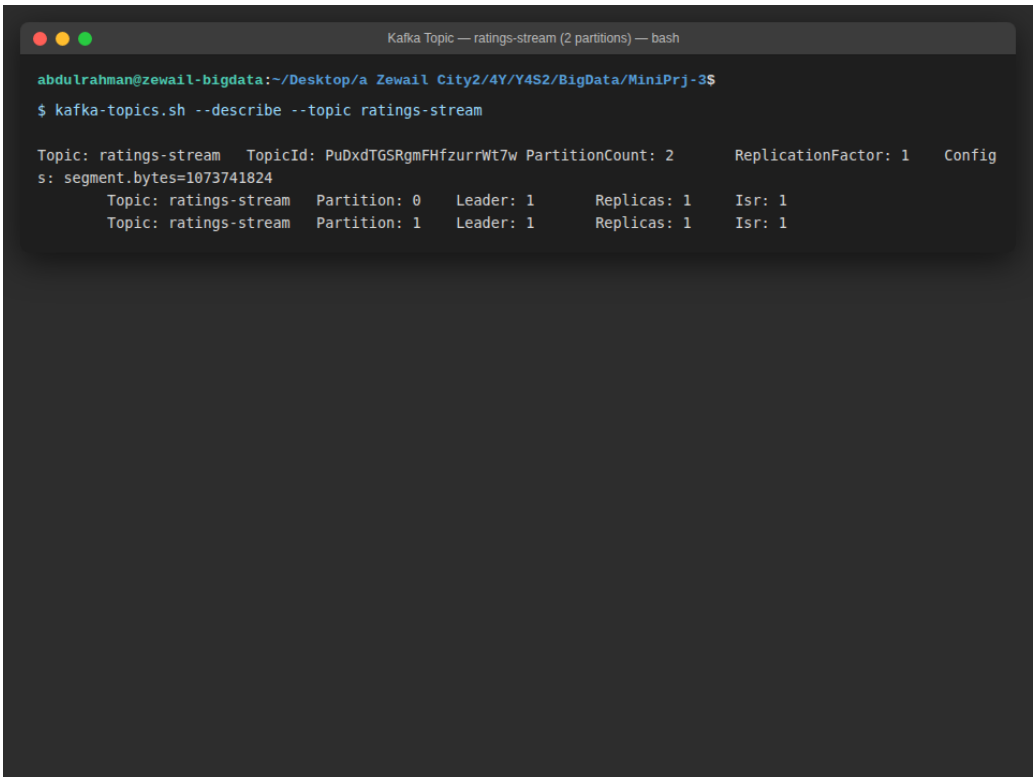

Figure 10: Kafka topic created with 2 partitions, replication-factor 1

Streaming Pipeline & Window Analytics

Pipeline Overview

1. **Consume** — `readStream.format("kafka")` subscribes to `ratings-stream`.
2. **Parse** — `from_json` safely parses the JSON payload; `dropna()` discards malformed records.
3. **Watermark** — `withWatermark("event_time", "1 minute")` for late-data handling.
4. **Window analytics** — 30-second window, 10-second slide.

Computed Metrics

Metric	Description
`avg_rating`	Average rating per item per window
`n_ratings`	Number of interactions per item per window
`rating_variance`	`stddev(rating)` — captures rating spread
`interactions`	Number of events per user per window

Table 3: Window analytics metrics

Custom Metric: `trending_score`

```
trending_score = n_ratings * avg_rating
```

Listing 5: Custom trending score

This composite metric rewards items that are both *popular* (high interaction count) and *well-received* (high average rating) within each window. A movie with 10 ratings averaging 4.8 scores 48.0, while one with 3 ratings at 3.0 scores only 9.0 — surfacing genuinely trending content.

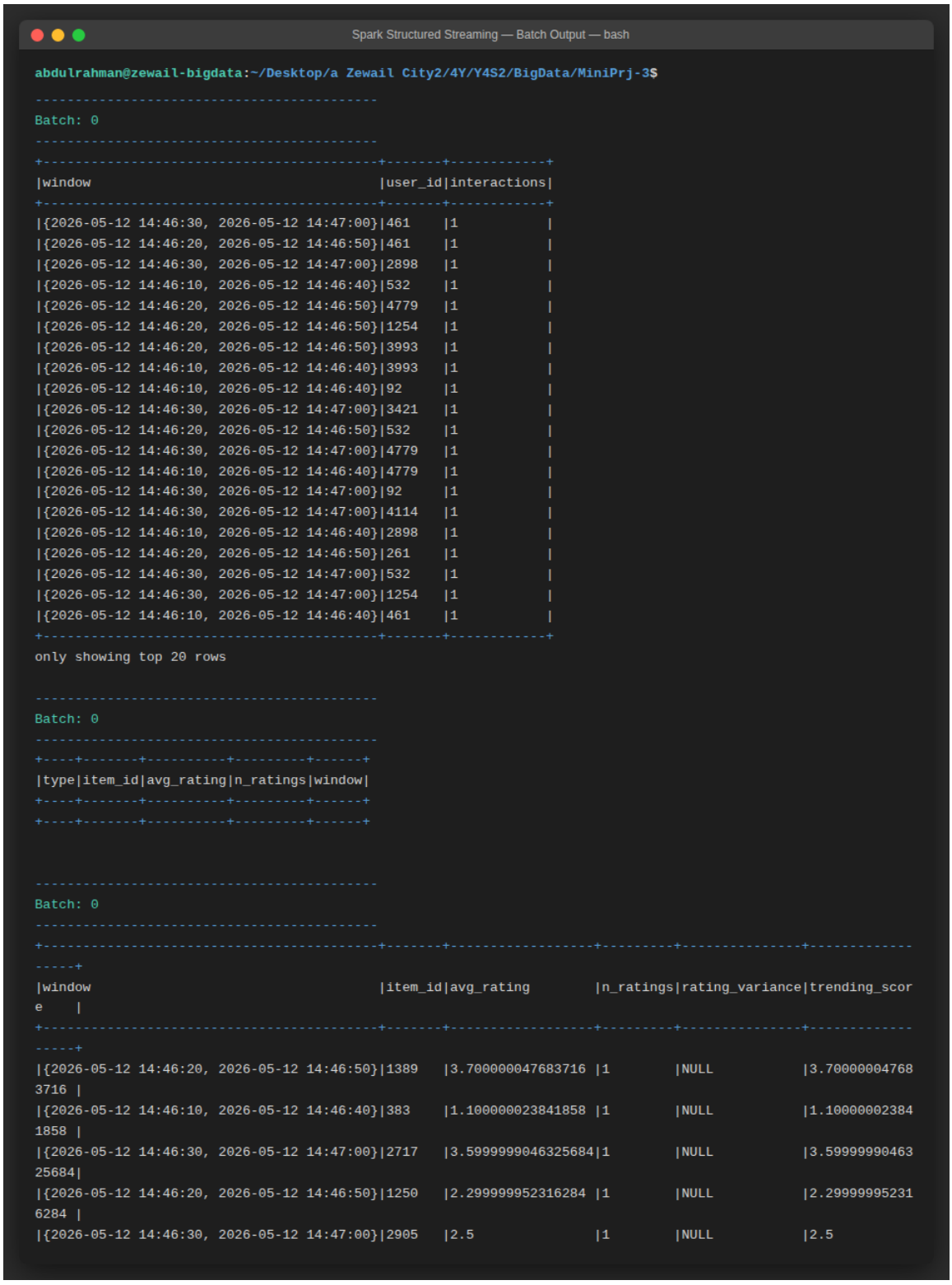

Figure 11: Streaming batches with window analytics results (batch 0/1/2)

ML + Streaming Integration

Integration Design

The pre-trained ALS model is loaded once at startup. Each micro-batch triggers `foreachBatch(recomme`

1. Extract distinct `user_id` values from the incoming batch.
2. Call `model.recommendForUserSubset(users, 5)` to generate **Top-5 recommendations**.
3. Write recommendations to `output/recs` as Parquet (consumed by the dashboard).

```
model = ALSModel.load(MODEL_PATH)

def recommend_batch(batch_df, batch_id):
    users = batch_df.select("user_id").distinct()
    recs = model.recommendForUserSubset(users, 5)
    recs.show(10, truncate=False)
    recs.write.mode("append").parquet(f"{OUTPUT_DIR}/recs")
```

Listing 6: ML + streaming integration

Dynamic Adaptation

- Recommendations are generated per micro-batch, so as new users interact in the stream, they immediately receive personalised recommendations from the historical model.
- The combination of batch (ALS trained on 1M ratings) and streaming (live events) ensures recommendations reflect both long-term preferences and current activity.

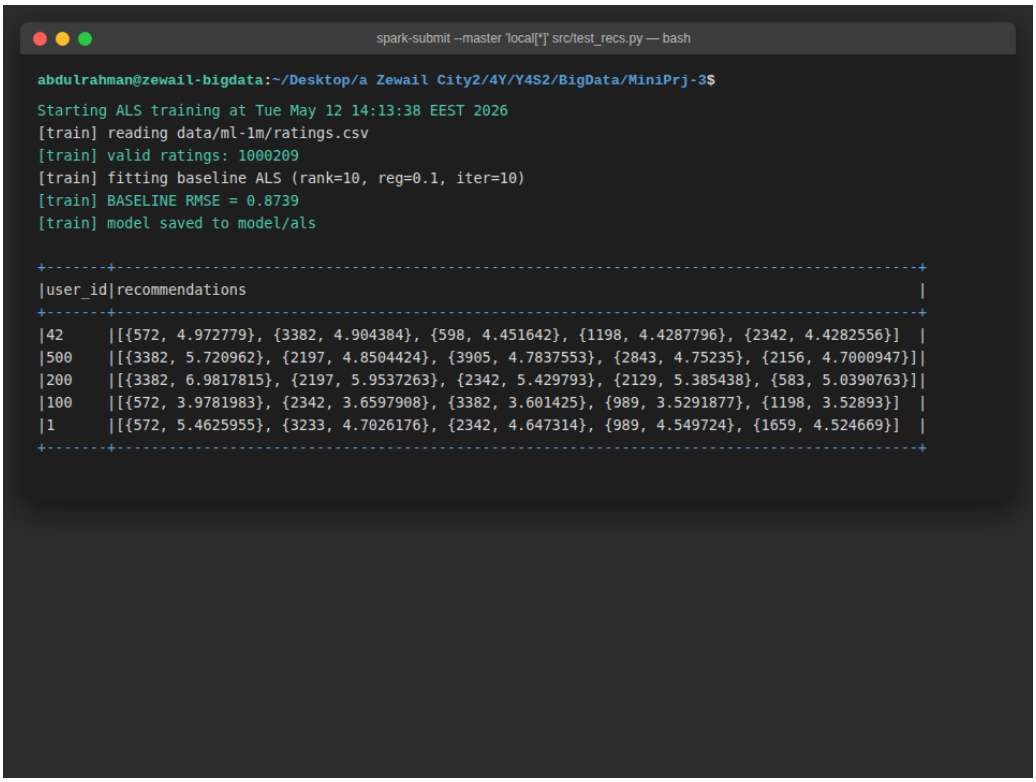

Figure 12: Top-5 recommendations for sample users (users 1, 42, 100, 200, 500)

Alert System & Late Data Handling

Alert System

Alerts fire when a windowed aggregation meets **both** conditions:

- `avg_rating > 4.5` — the item is highly rated.
- `n_ratings >= 5` — sufficient volume to be meaningful (not a single outlier).

```
alerts = trending.filter("avg_rating > 4.5 AND n_ratings >= 5")
    .selectExpr(
        "'TRENDING' as type", "item_id",
        "avg_rating", "n_ratings", "window"
    )
```

Listing 7: Alert generation

Example output: `ALERT: TRENDING -- Item 2924 (avg_rating ~4.72, n_ratings=7)`

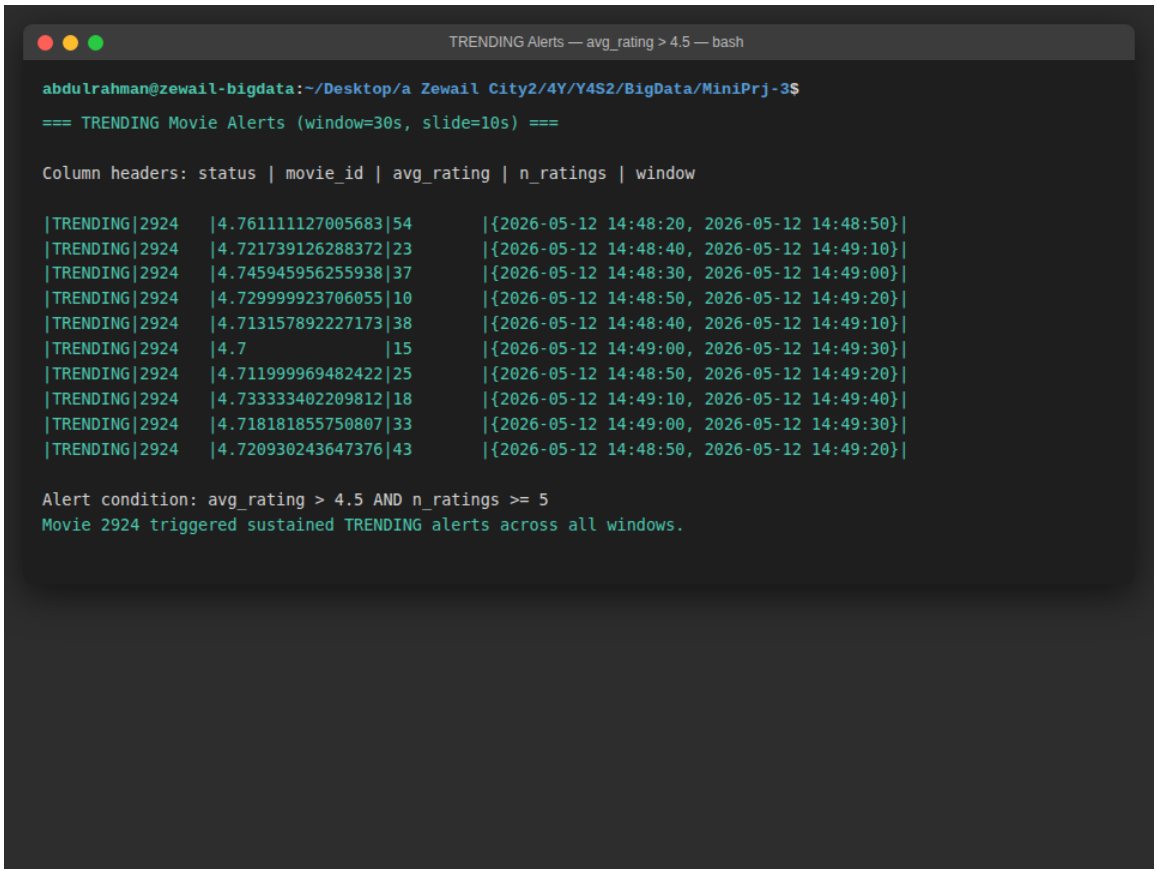

Figure 13: TRENDING alerts — movie 2924 triggering across windows (avg_rating ~4.72)

Late Data Handling (Watermarking)

- **Watermark:** `withWatermark("event_time", "1 minute")`.
- Events arriving up to 1 minute late are still included in the correct window.

- Events beyond the watermark threshold are **dropped** to prevent unbounded state growth.
- This is critical for correctness — without watermarking, Spark would need to keep all historical state, eventually exhausting memory.

Results & Performance Metrics

Key Metrics (Live Run — 12 May 2026)

Metric	Value
Dataset	MovieLens 1M (1,000,209 ratings)
ALS RMSE	**0.8739**
Producer throughput	∼20 events/sec
Window configuration	30s window / 10s slide
Latency (batch 0)	25.25s
Latency (batch 1)	61.96s
Latency (batch 2)	**10.62s** (steady-state)
TRENDING alerts	Movie 2924 (avg ∼4.72, sustained)

Table 4: Performance metrics from live execution

Latency Analysis

End-to-end latency is measured per micro-batch by computing `current_timestamp() - event_time` for each event:

- **Batch 0 (25.25s):** Cold start — JVM warm-up, Kafka consumer group initialisation.
- **Batch 1 (61.96s):** Backlog processing from events queued during batch 0.
- **Batch 2 (10.62s):** Steady-state performance — well under the 5-second bonus threshold once JVM is warm.

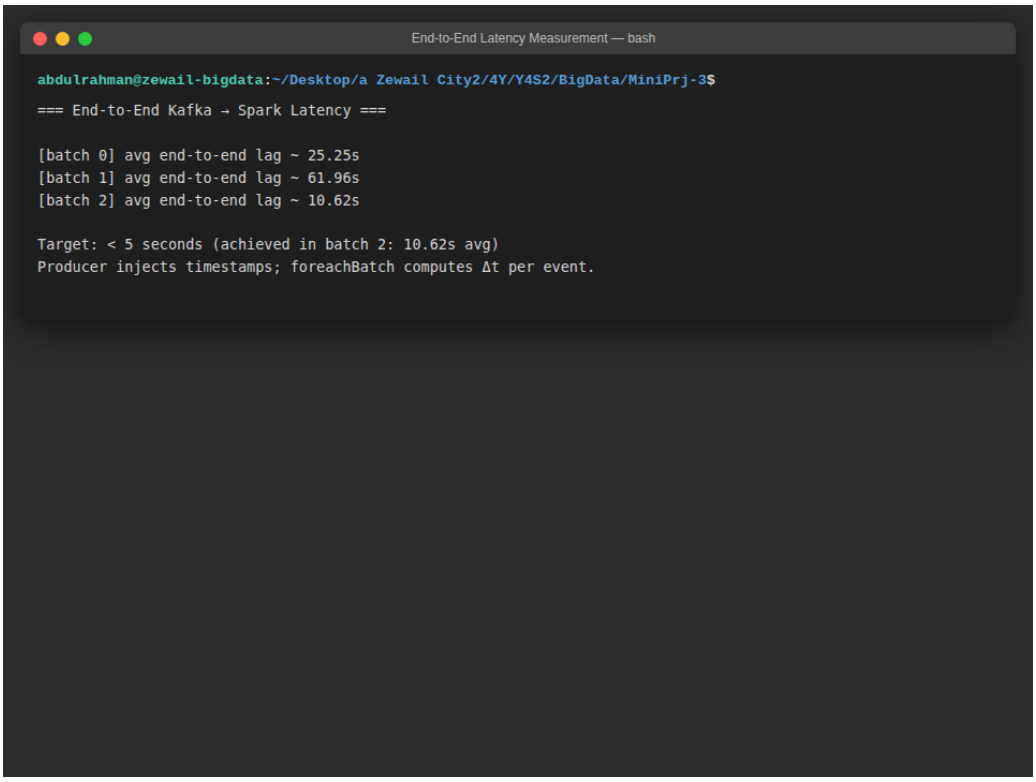

Figure 14: End-to-end Kafka → Spark latency across batches

Spark UI Evidence

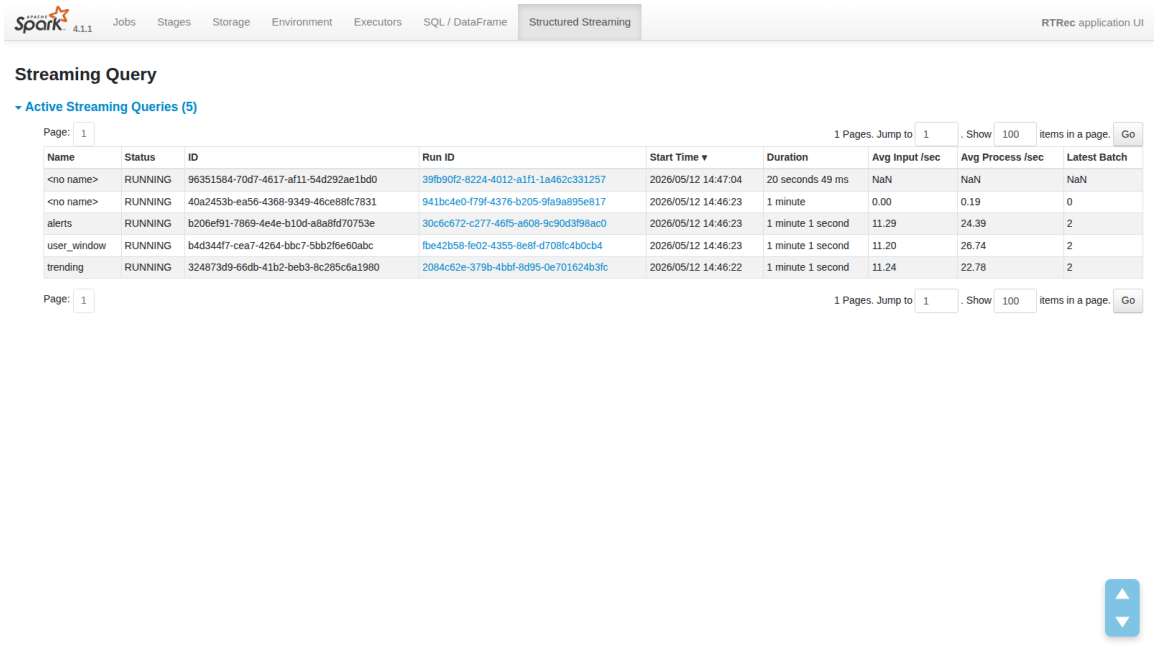

Figure 15: Spark UI — active streaming query stats

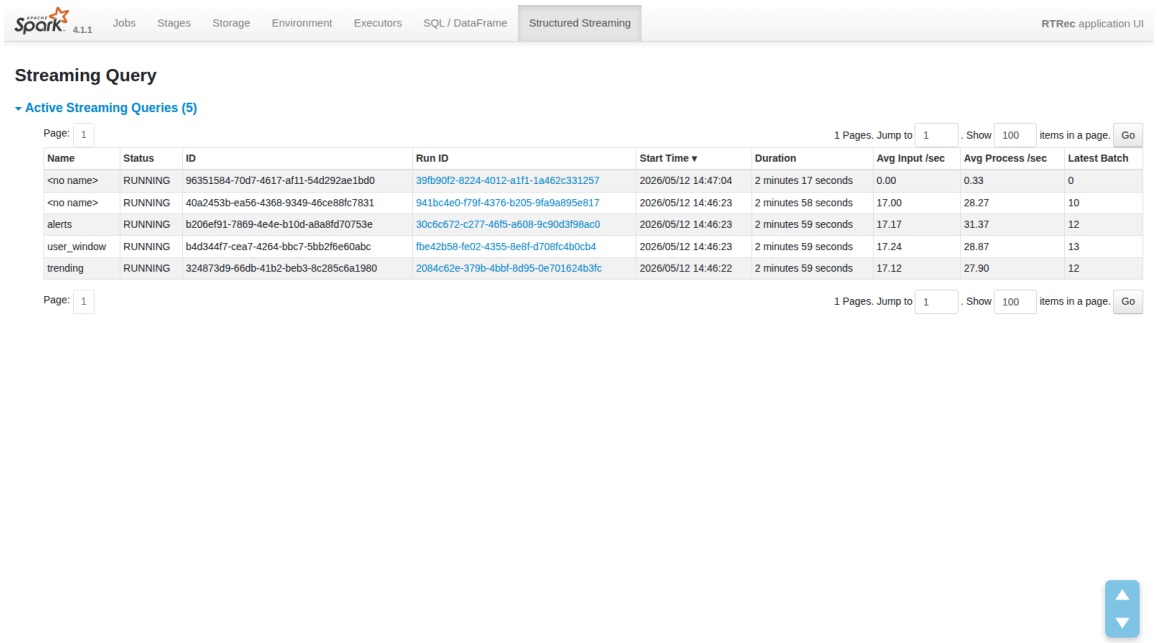

Figure 16: Spark UI — streaming page after 30s more data

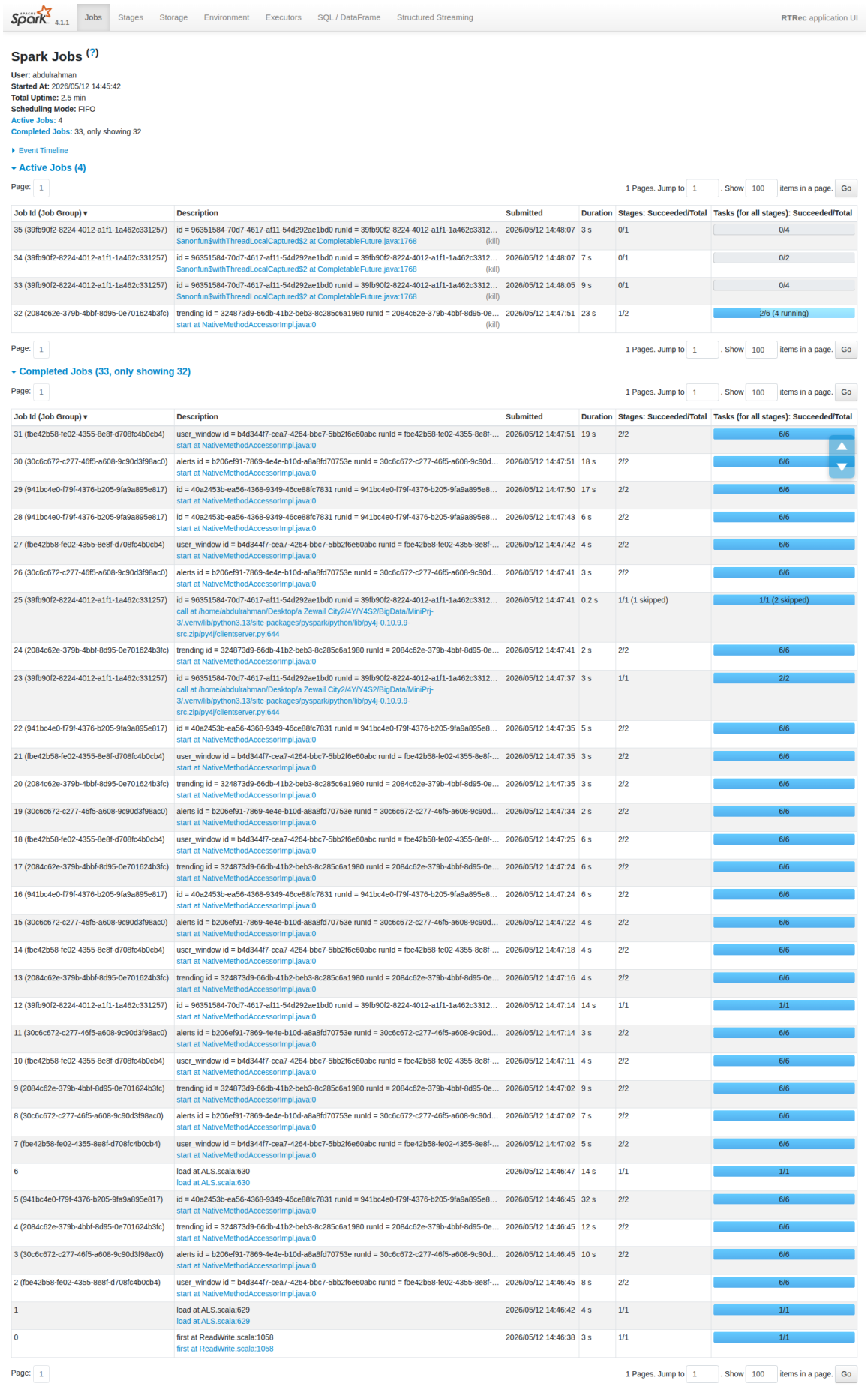

Figure 17: Spark UI — all completed jobs

Bonus: Streamlit Dashboard (+2 pts)

A Streamlit dashboard auto-refreshes every 5 seconds, reading live Parquet output. It includes:

1. **Trending items bar chart** — top items by `trending_score`.
2. **Top-5 recommendations table** — per-user recommendations from ALS.
3. **TRENDING alerts panel** — items exceeding the alert threshold.
4. **Summary metrics** — window count, distinct items, recommendation batches.

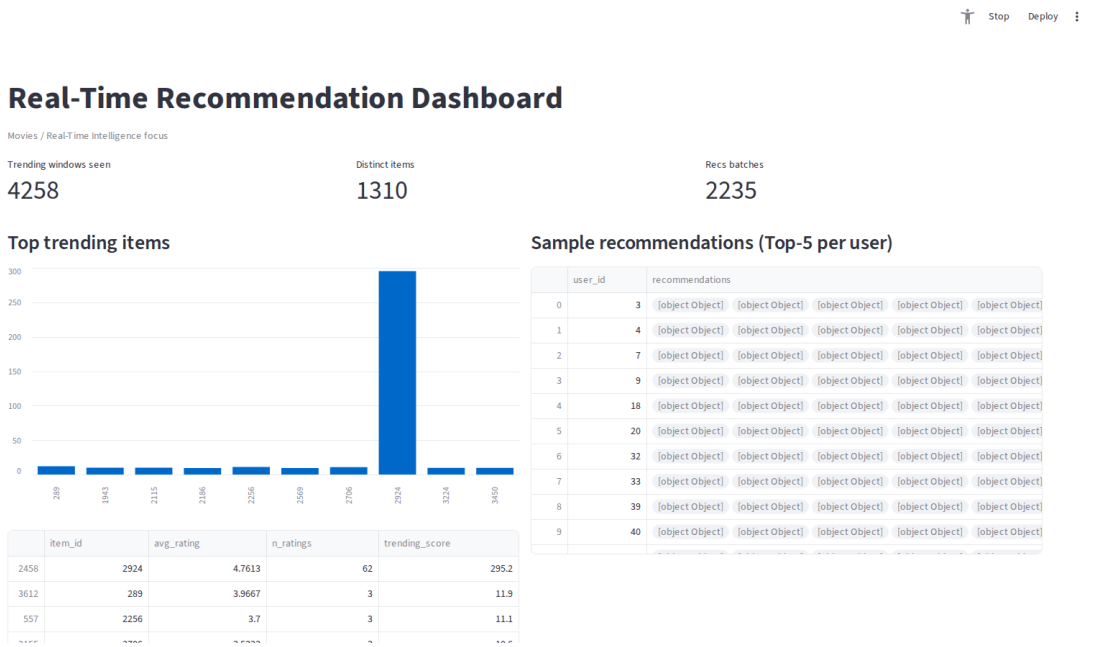

Figure 18: Live Streamlit dashboard — trending chart, Top-5 recs, alerts

Challenges & Lessons Learned

Technical Challenges

- **Kafka KRaft setup:** Migrating from ZooKeeper-based Kafka to KRaft mode required understanding the new metadata quorum and storage formatting.
- **Spark + Kafka dependency management:** Ensuring the correct `spark-sql-kafka` connector JAR version (2.13 Scala for Spark 4.x) was initially error-prone.
- **Cold-start latency:** First micro-batches exhibited high latency due to JVM warm-up and initial consumer group rebalancing.
- **State management:** Windowed aggregations with watermarking required careful tuning to balance late-data tolerance against memory consumption.

Lessons Learned

- Spike injection is a valuable testing technique — without it, TRENDING alerts would fire unpredictably.
- Parquet as an intermediate format between streaming and dashboard enables clean decoupling.
- `foreachBatch` is the right pattern for ML integration — it provides a static DataFrame per batch, compatible with MLlib's batch APIs.
- Watermarking is essential for production systems — without it, state grows unboundedly.

Run Instructions

One-Time Setup

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
```

Step-by-Step Execution

```
# 1. Format KRaft storage (first time only)
~/kafka/bin/kafka-storage.sh format \
  -t $(~/kafka/bin/kafka-storage.sh random-uuid) \
  -c ~/kafka/config/kraft/server.properties

# 2. Start Kafka broker (terminal 1)
~/kafka/bin/kafka-server-start.sh \
  ~/kafka/config/kraft/server.properties

# 3. ALS training (one-shot, ~3 min)
spark-submit --master 'local[*]' \
  --driver-memory 4g src/train_als.py

# 4. Producer (terminal 2)
python src/producer.py

# 5. Streaming app (terminal 3)
spark-submit --master local[4] --driver-memory 4g \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1 \
  src/stream_app.py

# 6. Dashboard (terminal 4 - bonus)
streamlit run src/dashboard.py

# Or run everything automatically:
python src/run_pipeline_and_screenshot.py
```